INSIGHTENSE
AI INSIGHTS WITH A SENSE FOR EXCELLENCE

Project Launch Runbook

Cowork multi-session project bootstrap

From “create a project” to the coordinator’s “team up & ready” report

For the Operator, Coordinator, and Curator

Edition EN · 2026-07-03 · Reusable, product-agnostic methodology

Document revision history

This table lists the revision history of this document.

Version	Date	Summary of changes	Product release ID	Shipped-with (commit/barrier)
1.0	2026-06-11	initial runbook	RTM-REL-2026.06	(pending push)
1.1	2026-06-12	command registry mirrored from sec10 (canonical); permanent role mailboxes; CC-prompt discipline	RTM-REL-2026.06	(pending push)
1.2	2026-06-13	NORM-CUR-01/04/05/06/06b/07/07b/07c/08/09a/09b/10; auto-inbox-hook (L-SC-27); L-CUR-01/02 + L-DEPLOY-01/02/03	RTM-REL-2026.06	(pending push)

Contents

Purpose & audience

This runbook captures the proven process for launching a new Cowork project as a multi-session team: one operator, one coordinator, an optional external curator, and a set of specialist sessions. It exists so a new project can reach a credible “team up & ready” state quickly and safely, with the same discipline that keeps a long-running project honest.

Audience: the Operator (directs the project), the Coordinator (owns the bus and the gates), the Curator (external source-of-truth / certifier), and specialists being onboarded.

How to use this runbook

- **Work the six phases P0 → P5 in order.** Each has Goal · Who · Actions · Artifacts · Verification gate · Operator GO.
- **Every gate is verified against the object store / repo walk**, never a chat claim.
- Copy the Appendix templates (A–E) and fill placeholders.The cross-cutting invariants in §5 are binding throughout.

1. Roles & responsibilities

Role	Mandate	Gate it holds
Operator	Directs the project; the only one who pushes and gives GO; schedules session turns.	Every phase GO.
Coordinator	Runs the bus and the process (§0, coordination, registry, .coord/, commit serialization, push barrier, ready report).	P1–P5 gates.
Curator	External SME / source capture; certifies “core raised” by an object-store walk.	P1 core-raised certification.
Specialists	Module owners (backend, frontend, DB, devops, …); do the work within their claims.	Their READY/HOLD at barriers.
Security	Mandatory cross-cutting gate.	Mandatory ACK before any prod release.
Tech Writer	Documentation gate and author.	Doc-sync impact-triage ack in the push quorum (NORM-CUR-10).

2. The six-phase model

```
P0 Universe -> P1 Core -> P2 Roster & Claims -> P3 Tree hygiene -> P4 Work-done map
-> P5 Ready
  (operator)    (coord)       (coord + operator)     (coord/CC)          (coord)
(coord)
```

3. Phases

P0 — Universe (the empty world)

Goal: a mounted, version-controlled project folder with a seed spec. **Who:** Operator.

- Create/connect one Cowork project folder; record its single absolute working path.
- git init; create the working branch; pick a short session prefix.
- Seed CLAUDE.md: product, stack, scope/out-of-scope, working-folder path, prefix.
- Commit the baseline.

Verification gate

git log shows the repo; the folder is mounted; CLAUDE.md present (object-store check). Operator GO → build the core.

P1 — Core (discipline + bus)

Goal: the binding §0 discipline, the coordination protocol, and the .coord/ bus exist. **Who:** Coordinator.

- Extend CLAUDE.md §0 (BINDING): 0.1 verify; 0.2 resume-integrity; 0.3 Python+os.fsync writes, Edit tool BANNED on the mount; 0.4 pre-commit check; 0.5 mount .git/status UNRELIABLE → object-store verify; 0.6 NO auto-push + you cannot push from the mount [L-SC-20]; 0.7 post-commit re-sync (PD-007).
- **Adopt the canonical session-coord skill (NORM-CUR-05, a P1 step):** copy the agnostic core into .claude/skills/session-coord/session-coord.md (operative source = §10). The CLAUDE.md command-registry is a POINTER to it; re-deriving a verb subset in CLAUDE.md is banned (it caused the RTM/AD divergence). Project bindings stay in CLAUDE.md; core changes propagate via L-SC-15 cache-bump.
- Add the multi-session coordination section + the command registry (pointer to §10; see Appendix F mirror).
- Build the .coord/ bus (Appendix B).
- Write one init-prompt per planned specialist (Appendix C); tag not-yet-active roles FROZEN / NOT-YET-EXISTS.

Verification gate

The curator certifies by object store (not chat) — §0 anchors present, bus files exist, init-prompts carry the mandatory blocks. Operator GO → raise the roster.

P2 — Roster & Claims

Goal: a validated, non-overlapping claim-map. **Who:** Coordinator (+ operator confirms the roster).

- Operator confirms the initial roster.
- Build the claim-map (Appendix D): module → paths, seam rules, READ-ONLY zones.
- Validate against the real tree: paths exist? nesting? overlaps?
- The roster is NOT frozen — re-validate on every operator change.
- Share a cross-cutting asset by OWN / REVIEW / APPLY across roles — never carve a sub-path out of an owner's claim.

Verification gate
Real paths, ZERO overlap, a single owner per seam. Operator GO → tree hygiene.

P3 — Tree hygiene

Goal: a clean, intentional, committed tree. **Who:** Coordinator issues CC tasks; specialists run them (native CC, not the mount).

- Commit .gitignore FIRST, then re-check status.
- Decide tracked vs ignored BEFORE any blanket add.
- De-duplicate documents ONLY by a verified subset proof (X subset of Y, zero unique).
- Commit existing work in labeled, module-grouped commits — native git / CC, serialized by commit.lock, journaled, post-commit re-sync.

Verification gate
git status clean; every commit journaled. Operator GO → work-done map.

P4 — Work-done map

Goal: an honest map of what actually exists. **Who:** Coordinator.

- Build the map by walking the committed tree, labeling Intended / Drafted / Delivered (verified) / Committed (hash).
- Delivered (verified) requires a green build/test — a commit is not “Delivered”.
- Distribute the first tasks per the claim-map (via the dispatch loop, §4).

Verification gate
The map is built from the repo, not chat; every “Delivered” is backed by a build/test. Operator GO → ready.

P5 — Ready

Goal: the team is up and the first work is moving. **Who:** Coordinator.

Write the “team up & ready” report (Appendix E) — a Definition-of-Done checklist including the claim-map and confirming each “Delivered” item was verified by the object store (git cat-file / show / hash-object), not chat, plus a defined push barrier + quorum (mandatory Security ack + Tech-Writer doc-sync ack).

4. Operational loop — task dispatch (the heartbeat after launch)

Once the team is stood up (P5), work moves operator-mediated: the coordinator routes, the operator triggers, the specialist executes.

1. **The coordinator drafts** a full self-contained directive — коорд: промпт <role> <task>: mandatory reads + integrity block + the claim + task + acceptance criteria + commit.lock/journal/no-push. Code points at a tools/<file>.md (CC-only rule).
2. **The coordinator writes** it into the specialist's role inbox .coord/inbox/<role>.md (router duty); it does not execute or trigger.
3. **The coordinator hands the operator a trigger-list** — the session slugs to activate, each annotated for parallelism (NORM-CUR-09a): || = may run in parallel (independent); -> = must run sequentially (output-dependency or a shared exclusive claim / commit.lock). Default -> when unsure.
4. **The operator runs the intake** (коорд: входящие) in each named session; that session reads its own inbox and does the CC work.
5. **The specialist reports back** to the coordinator; a handled-marker is written only by the recipient.
6. **The coordinator** on its next intake reads its inbox, journals the outcome, routes the next step.

Execution leg — CC↔spec binding (NORM-CUR-07 / 07b / 07c). The specialist↔CC handoff runs on a dedicated binding channel .coord/cc/<role>.md. Binding-open is MANDATORY in the §0.6a CC-prompt template — every CC prompt opens the binding after the integrity block and writes a RESULT (commits / build-test / files / status / blockers, object-store verified) at the end; коорд: промпт enforces both. The spec consumes the RESULT (> consumed) and relays a digest to the coordinator. Write-routing: CC results land in the binding natively, not operator-relayed (operator-relay = backstop for no-repo/server contexts); small chatter stays Cowork Python+fsync. The binding is an INDEX to git, not a source of truth — if a RESULT is absent but git log shows commits, the spec reconciles from the object store (L-SC-04/L-SC-18 fallback).

Auto-inbox-hook (SAFE default, best-effort — L-SC-27). Each session self-attends its permanent role inbox every turn without an explicit коорд: входящие: a cheap turn-start peek, idle auto-process, mid-task defer, completion prompt. It is best-effort, NOT autonomous — the peek runs only when the session is given a turn (no daemon, no polling) and does NOT replace the explicit коорд: входящие / коорд: статус triggers (the reliable path; the operator stays the scheduler). A block arriving while a session has no turn is not seen until the next poke — expected, not an anomaly. The hook also triggers inbox auto-archival (NORM-CUR-06) when oversized.

Invariants of the loop

All code changes go out as CC prompts (CC-only rule); the coordinator analyses/plans/reviews/journals only. A handled-marker is written only by the recipient. Commits run under commit.lock, are journaled, and never auto-push.

5. Cross-cutting invariants (binding throughout)

- **Verify, not chat** — tool success ≠ delivery; build status tables by walking the repo (§0.1).
- **Python + os.fsync for every write; the Edit tool is BANNED on the mount** (§0.3).
- **Object-store verification** — the mount's .git / git status is unreliable; confirm with git hash-object vs git rev-parse HEAD:<file>; never escalate "corruption" from a mount read (§0.5).
- **No auto-push; you cannot push from the mount** — every push is a conscious operator decision via the dedicated push prompt [L-SC-20] (§0.6).
- **Post-commit re-sync from HEAD** — counteract the mount's async cache write-back (PD-007, §0.7).
- **commit.lock covers the plumbing path too** — the commit-tree + direct ref-write path has no git-level locking; never bypass commit.lock.
- **Handled-markers are written only by the recipient** of a message.
- **CC-only code rule.** Every CC task prompt is a .md file under tools/, issued as a code box — git-versioned, §4-reviewable, no chat truncation. Project skills (.claude/skills/**) are edited via a CC prompt (NORM-CUR-03): native CC commits the versioned file, no push; they are normal versioned files, not read-only (that caveat is only the Cowork plugin cache); running sessions pick up an edit via the L-SC-15 cache-bump.
- **Documentation governance** — a coordinator-assigned Product Release ID RTM-REL-YYYY.MM[.patch] + a Shipped-with column; a doc-sync impact-triage gate in the push quorum; and the approved/{doc,pdf} + editing/ layout with a mandatory revision-history table.
- **Uniformity across all projects (NORM-CUR-01)** — every protocol/discipline decision and bus-norm change applies identically to all projects; the curator keeps them at parity (registry, mailbox norms, §0 discipline, push-barrier rules, this runbook are apply-to-all).
- **Anti-staleness of reference artifacts (NORM-CUR-08)** — any artifact mirroring live/generated state (db/functions, schema.sql, deploy docs) is generated/exported from its authoritative source and carries a 'generated from X at <ts>' provenance line, never hand-maintained; a freshness-audit (folded into коорд: разбери / проверь шину) flags drift; a clean from-scratch rebuild is the backstop truth-test; verify environment facts from the server, not the doc (L-DEPLOY-01/02/03).
- **Mandatory documentation by the Tech Writer (NORM-CUR-10, permanent)** — every decision/change/addition is always documented by the Tech Writer; nothing is 'done' until its docs are current; enforced at the push-barrier doc-sync gate and as a standing between-barrier duty; the coordinator routes every change to the Tech Writer; the curator verifies doc-coverage at filing.
- **Untracked ≠ backed up** — a deliverable that lives beyond one session is committed as a WIP on the branch immediately (docs: commit, no push); untracked/HELD files have no object-store backstop and are wiped by git clean (incident 2026-07-03).

6. Case study & lessons — Agent Desktop bootstrap

Worked example: Agent Desktop / Bynet — a Cisco UCCE/UCCX agent desktop via Finesse; .NET 8 backend, frontend later. Captured by the curator; the normative steps are §1–§5 above.

Lessons (generalize for any project)

7. The roster is not frozen — re-validate the claim-map for overlap on every operator change.
8. Share a cross-cutting asset by OWN / REVIEW / APPLY, never by carving a sub-path out of an owner's claim.
9. De-duplicate documents only by a verified subset proof, never by assumption.
10. The mount cannot commit/push (stuck .git locks; bindfs denies rm/rename in .git) — all commits run native CC, serialized by commit.lock, journaled, with a post-commit re-sync [L-SC-20].
11. Certify "core raised" by walking the repo via the object store, never from a chat claim.
12. A handled-marker is written only by the recipient.
13. A cross-project channel that depends on both mounts breaks on remount — route each direction through a file in the writer's own bus.

Discipline lessons — SQL, quoting & deploy freshness (apply on every project)

- **L-CUR-01 — quoted-content write doubling.** Writing SQL or any single-quoted content via Python string embedding can double apostrophes → a psql syntax error. Write SQL/quoted content via a quoted heredoc straight to file, never via Python string embedding; add a byte gate before packaging.
- **L-CUR-02 — unquoted mixed-case PostgreSQL identifier folds to lowercase.** WHERE TenantId becomes tenantid → "column does not exist". Any SQL touching mixed-case columns/tables must double-quote the identifier; a deploy/test gate greps for unquoted mixed-case identifiers.
- **L-DEPLOY-01/02/03 — deploy reference freshness.** Verify environment facts from the server, never the doc (01); a clean from-scratch rebuild is a truth-test that exposes stale mirrors a live server masks — run it periodically (02); hand-maintained mirrors (db/functions, schema.sql, deploy docs) drift silently → generate them from the authoritative source (Export-All / pg_dump), never hand-edit (03).

Appendix A — CLAUDE.md skeleton

```
# <PROJECT> — CLAUDE.md
## 0. Environment rules — executor must read first (BINDING)
### 0.1 Verification — tool success != delivery
### 0.2 Session-resume integrity  ### 0.3 Writes: Python+os.fsync ONLY; Edit tool
BANNED on the mount
### 0.4 Pre-commit check  ### 0.5 Mount .git/status UNRELIABLE -> object-store
verify
### 0.6 No auto-push; you cannot push from the mount [L-SC-20]
### 0.6a CC<->spec binding is MANDATORY in every CC prompt (NORM-CUR-07c) —
open .coord/cc/<role>.md after the integrity block; write the RESULT at the end.
### 0.7 Post-commit re-sync from HEAD (PD-007)
## <N>. Multi-session coordination — .coord/ bus; commit.lock; journal; push barrier
+ quorum (Security + Tech-Writer).
## <N+1>. Command registry — POINTER to the session-coord skill §10 (operative
source). Re-deriving a verb subset here is BANNED; see the mirror in Appendix F.
```

Appendix B — .coord/ bus files

```
.coord/
  README.md   .gitignore   sessions/<slug>.md   locks/commit.lock   journal.md
  inbox/<role>.md         # PERMANENT role mailbox (coordinator, curator, backend,
frontend, security, dba, devops, techwriter)
  inbox/archive/<role>.md # durable archive — role mailboxes self-prune (NORM-
CUR-06) via tools/inbox_archive.py
  cc/<role>.md            # CC<->spec binding channel (NORM-CUR-07)
  coordinator_handoff.md  push/request.md  push/acks/<slug>.md
```

Inbox auto-archival (NORM-CUR-06 / 06b). tools/inbox_archive.py keeps the last 25 blocks + last 24h in inbox/<role>.md and MOVES older blocks to inbox/archive/<role>.md (append-only, never deletes). Triggered by the auto-inbox-hook when oversized, or коорд: разбери / коорд: чистка. Concurrency-safe: the caller holds commit.lock; the script re-reads before write and writes via temp + os.replace (L-SC-09).

Appendix C — Init-prompt template (per specialist)

```
<PREFIX>: you are the standing <ROLE> specialist. Session name "<Name>".
0. COMMS: inbox = .coord/inbox/<role>.md (PERMANENT). Outbox = inbox/coordinator.md.
All writes Python+os.fsync.
1. INTEGRITY (0.2): git status; for M files hash-object vs HEAD; restore truncated.
2. MATERIAL: CLAUDE.md §0 + coordination + your domain sections.
3. TERRITORY: you OWN <paths>; READ-ONLY elsewhere; shared seams via
OWN/REVIEW/APPLY.
4. REGISTER: write .coord/sessions/<slug>.md, status active.  5. READ BUS: inbox;
sessions/*; journal; push/request.
6. PUSH QUORUM: READY/HOLD at every barrier.  7. CONFIRM: flush an ack to
inbox/coordinator.md + a short chat reply.
```

Appendix D — Claim-map template

```
# Claim-map — <PROJECT>
| Module | Owner (slug) | Paths | Seam rule | Read-only |
Validation: every path exists; ZERO overlap; one owner per seam. Re-validate on any
roster change.
```

Appendix E — “Team up & ready” report template

```
# Team up & ready — <PROJECT> (<date>)  — Definition of Done:
- [ ] Core: CLAUDE.md §0 (0.1-0.7) + coordination + registry — object-store
verified.
- [ ] Bus: .coord/ files present.  - [ ] Roster: sessions registered; claim-map;
ZERO overlap.
- [ ] Tree clean; commits journaled.  - [ ] Work-done map; each Delivered object-
store verified.
- [ ] First tasks issued.  - [ ] Push barrier + quorum (MANDATORY Security + Tech-
Writer doc-sync).  Operator GO: ____
```

Appendix F — Coordination command registry (mirror of session-coord §10)

Mirror — not an independent source

Operative source of truth = the project's session-coord §10. This list is a synchronized MIRROR for readers, never a second source. коорд: ревью and коорд: промпт are canonical.

Governing rules: (1) single source = §10 — every mirror POINTS to it, never a partial subset; (2) unknown verb → never guess → ask the operator; (3) all .coord/ writes Python+os.fsync, handled-marker only by the recipient; (4) operator replies are short.

Command	Action
коорд: ты координатор	Register as coordinator, read full bus; on a fresh session read the handoff + reconstruct.
коорд: статус	Own state; [coord] = full bus audit. Operator opens every RETURN with this (forced object-store resync).
коорд: входящие	Read own role inbox; act on each block; mark handled; auto-flush one response to inbox/coordinator.md.
коорд: сбрось	Flush status/question/handoff: update session file + append to the recipient's inbox.
коорд: разбери / проверь шину	[coord] reconcile inboxes+sessions (journal↔git), integrity sweep + fix.
коорд: чистка [<role>] (alias сессия: чистка)	Run inbox auto-archival (keep last 25 + 24h) → inbox/archive/<role>.md.
коорд: готовим пуш / дай ack / пуш	[coord] freeze barrier / write READY-HOLD / verify quorum (Security + Tech-Writer) → push prompt.
коорд: ревью <prompt> / коорд: промпт <role> <task>	[coord] §4-review a prompt/result; draft a full directive into inbox/<role>.md + hand the trigger-list.
коорд: handoff / передай координацию	[coord] produce a verified resume-checkpoint / hand off coordination.

Permanent role mailboxes

A session's mailbox (inbox/<role>.md) survives handoff/takeover — no migration, no lost directives (L-SC-21 fix). Session files stay slug-dated; only the mailbox is role-permanent.

Flush vs pause (NORM-CUR-04). коорд: сбрось (session form сброс=статус) means FLUSH status/question/handoff to the bus. There is no pause verb; the operator pauses via plain language. сбрось is not pause.

п.N == §N (NORM-CUR-09b). The operator keyboard has no §; "пункт N" is the semantic equivalent of section N — treat them identical.

Sources: *in-repo CLAUDE.md §0 / §42 / §44, the live .coord/ bus, and the session-coord skill (normative); the Agent Desktop bootstrap capture (case study). EN canonical.*